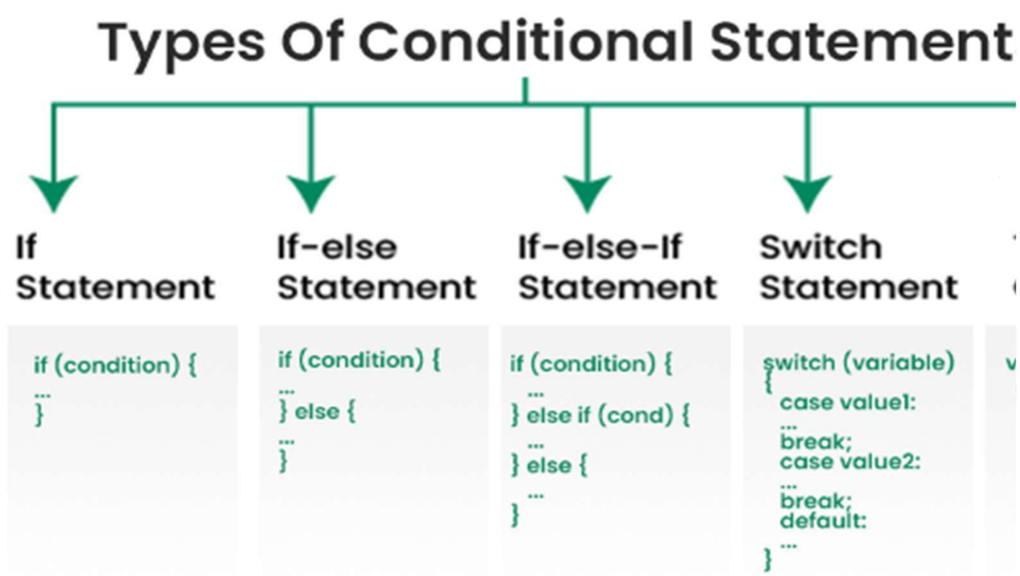
Chapter: 3

Decision Making in PHP

Conditional Statements

Conditional statements in programming are used to **control the flow of a program** based on certain conditions. These statements allow the execution of different code blocks depending on whether a specified condition evaluates to true or false.



1. If Conditional Statement:

The if statement is the most basic form of conditional statement. It checks if a condition is true. If it is true, the program executes a block of code.

Syntax of If Conditional Statement:

```
if (condition) {
   // code to execute if condition is true
}
```

if condition is true, the if code block executes. If false, the execution moves to the next block..

Example:

```
<?php
 $average=40;
  if($average>40)
    {
      print "You are selected";
    }
       print "you are not selected";
?>
```

2. **If-Else Conditional Statement:**

The if-else statement extends the if statement by adding an else clause. If the condition is false, the program executes the code in the else block.

Syntax of If-Else Conditional Statement:

```
if (condition) {
   // code to execute if condition is true
} else {
   // code to execute if condition is false
}
```

if condition is true, the if code block executes. If false, the execution moves to the else block.

```
<?php
        $average=40;
         if($average>40)
           {
             print "You are selected";
           }
        else
            {
              print "you are not selected";
            }
  ?>
```

3. **if-Else if Conditional Statement**:

The if-else if statement allows for multiple conditions to be checked in sequence. If the if condition is false, the program checks the next else if condition, and so on.

Syntax of If-Else if Conditional Statement:

```
if (condition1) {
   // code to execute if condition1 is true
} else if (condition2) {
   // code to execute if condition2 is true
} else {
   // code to execute if all conditions are false
}
```

In else if statements, the conditions are checked from the top-down, if the first block returns true, the second and the third blocks will not be checked, but if the first if block returns false, the second block will be checked. This checking continues until a block returns a true outcome.

```
<?php
        $average=40;
         if($average==40)
           {
             print "You are selected";
           }
        elseif($average<40)
```

```
            {
              Print “you are not selected”;
            }
        else {
            Print “You are selected”;
}
  ?>
```

The switch statement is used when you need to check a variable against a series of values. It’s often used as a more readable alternative to a long if-else if chain.
In switch expressions, each block is terminated by a break keyword. The statements in switch are expressed with cases.

Switch Conditional Statement Syntax:

```
switch (variable) {
    case value1:
        // code to execute if variable equals value1
        break;
    case value2:
        // code to execute if variable equals value2
        break;
    default:
        // code to execute if variable doesn't match any value
}
```

Use Cases of Switch Statement:

- Selecting one of many code blocks to execute based on the value of a variable.
- Handling multiple cases efficiently.

```
<?php
           $grade = “A”;
           Switch($grade== “A”)
            {
                 case($grade== ‘A’):
                    print “I group selected”;
                    break;
                 case($grade == “B”):
                     print “II group selected”;
                     break;
                  case($grade== “C”):
                     print “III group selected”;
                     break;
                  default:
                      print “No group selected”;
                      break;
              }
    ?>
```